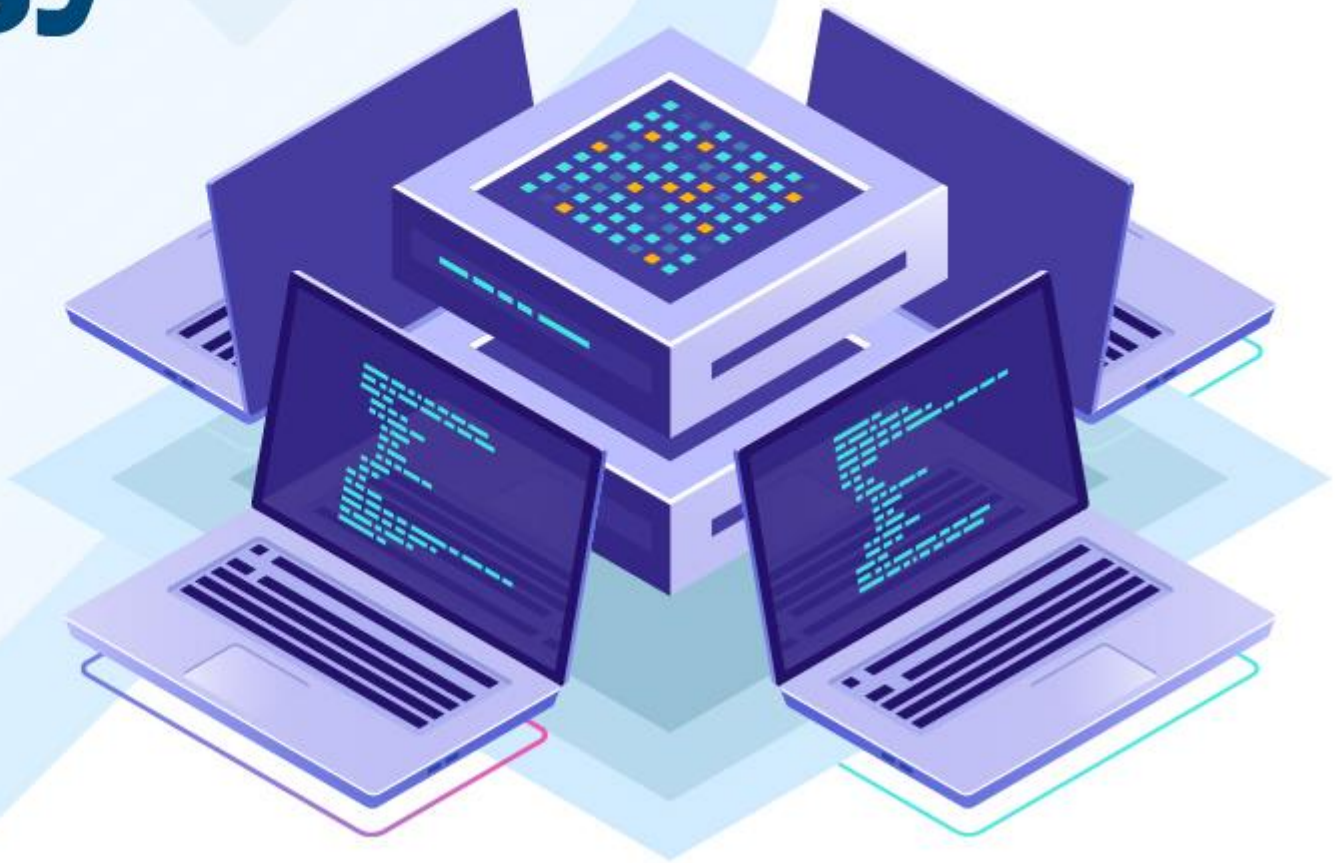
Information Technology

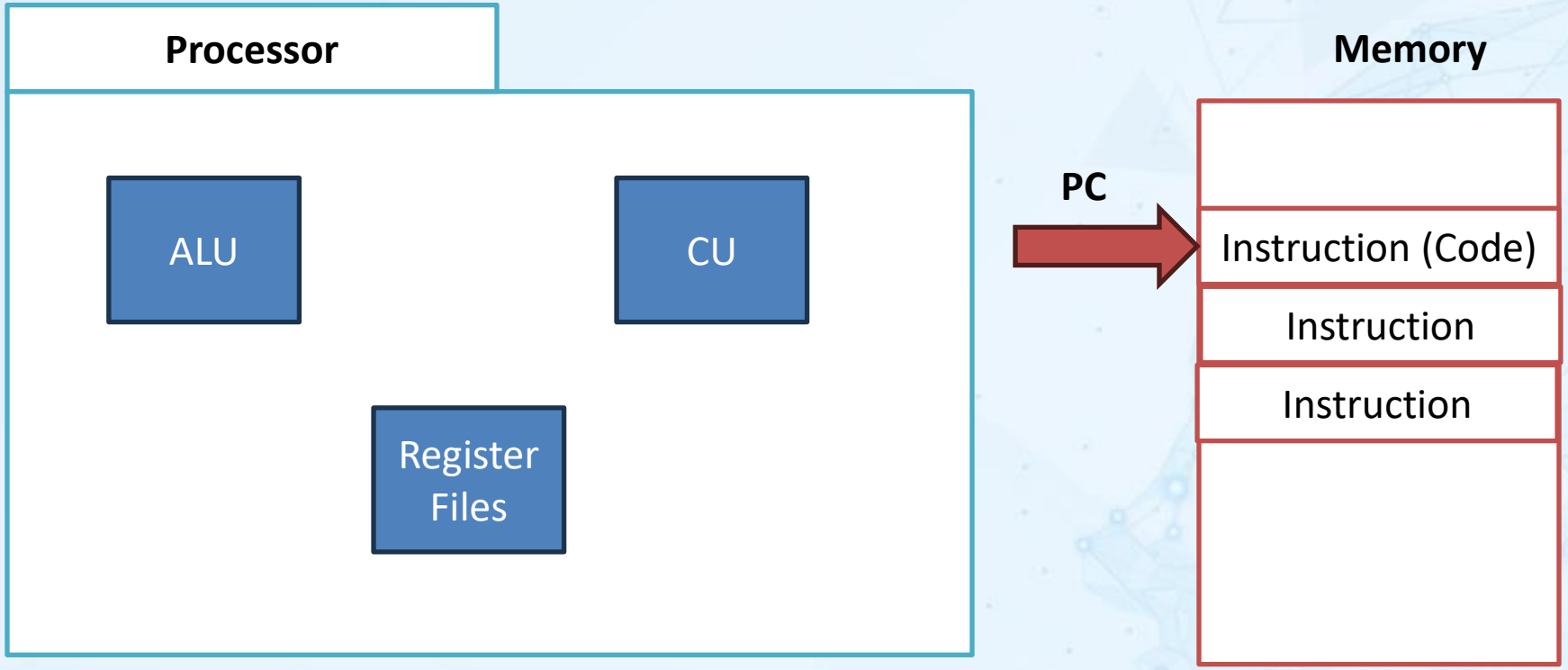
Fall Semester
2026 - 2027

- Embedded Systems
- **Dr. Yasmine Mahmoud**



EELU
الجامعة المصرية للتعلم الإلكتروني الأهلية
THE EGYPTIAN E-LEARNING UNIVERSITY

Instruction Life Cycle (How?)

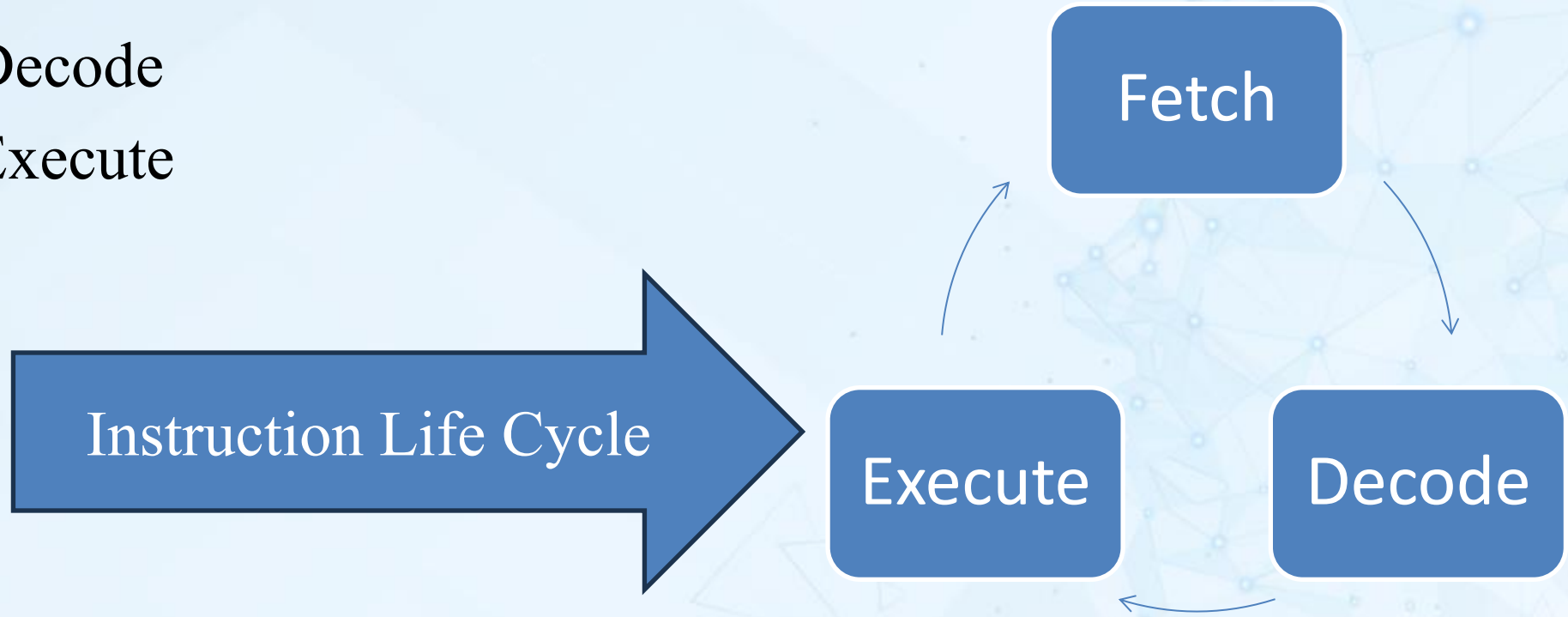


- PC is the program counter
- PC is a register in register files
- How PC knows its first (initial) location? Using startup code (later)

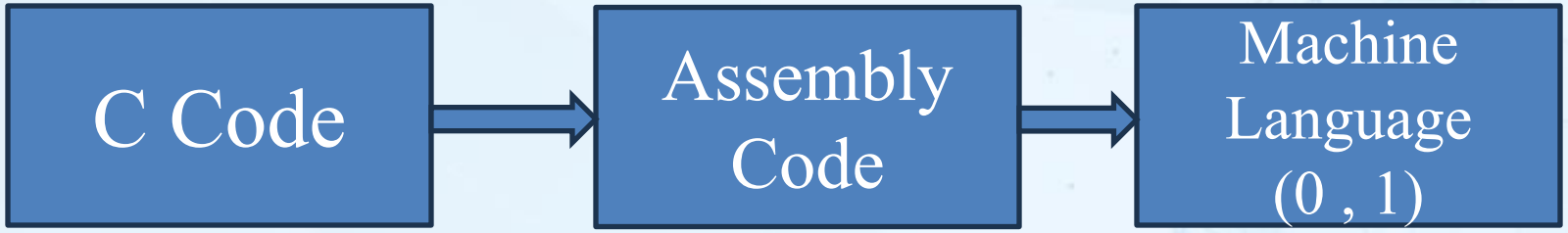
EELU

1. Processor

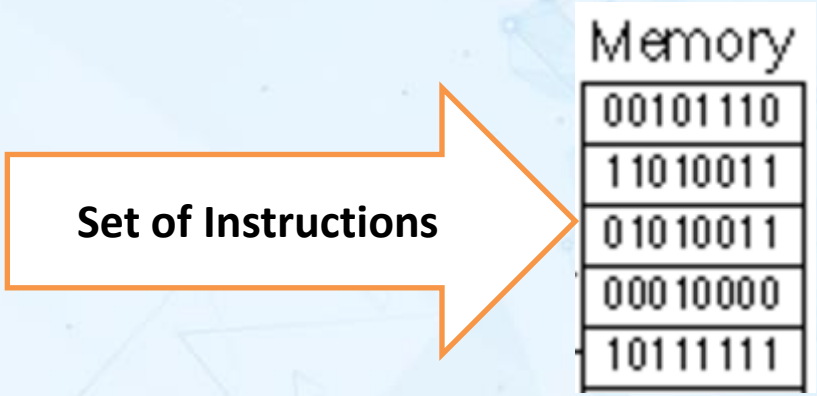
- Role of the processor is making three main things:
  - Fetch
  - Decode
  - Execute



Instruction

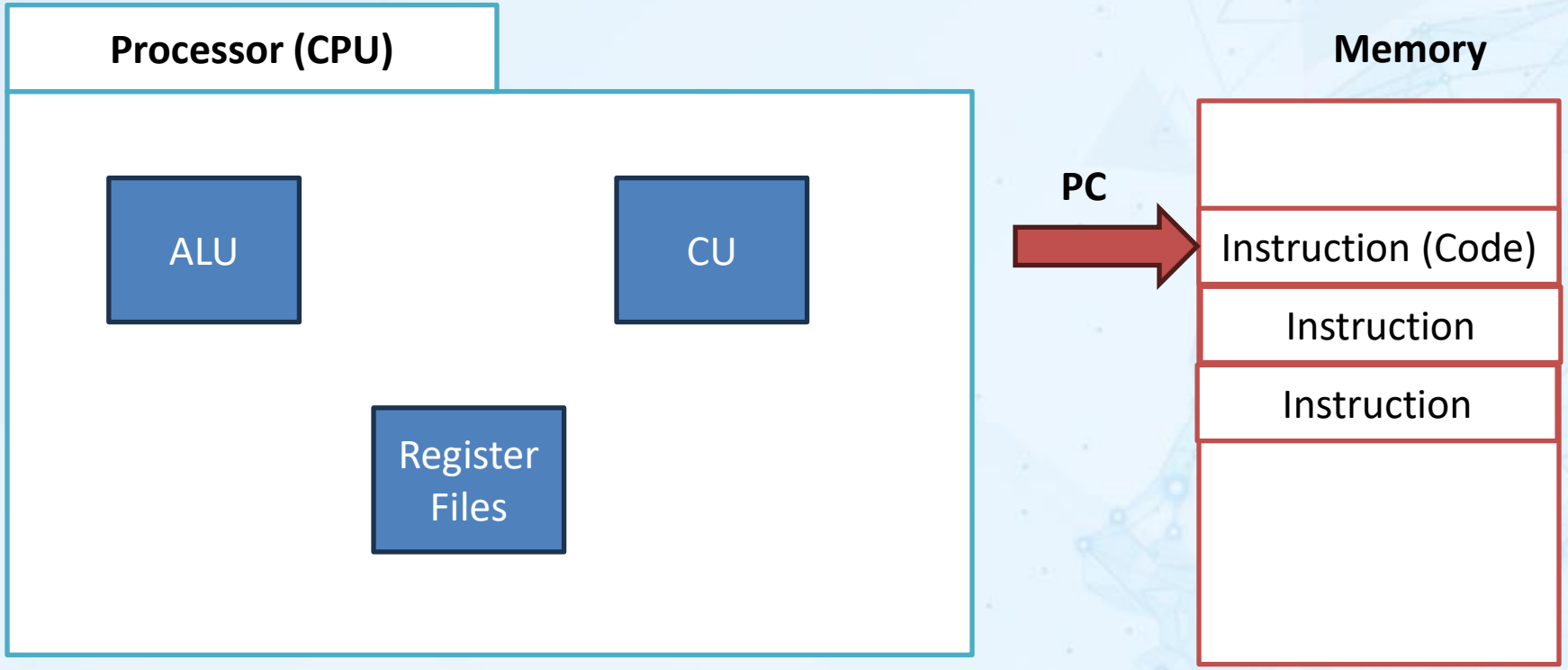


- 0,1 number of instructions stored in ROM



EELU

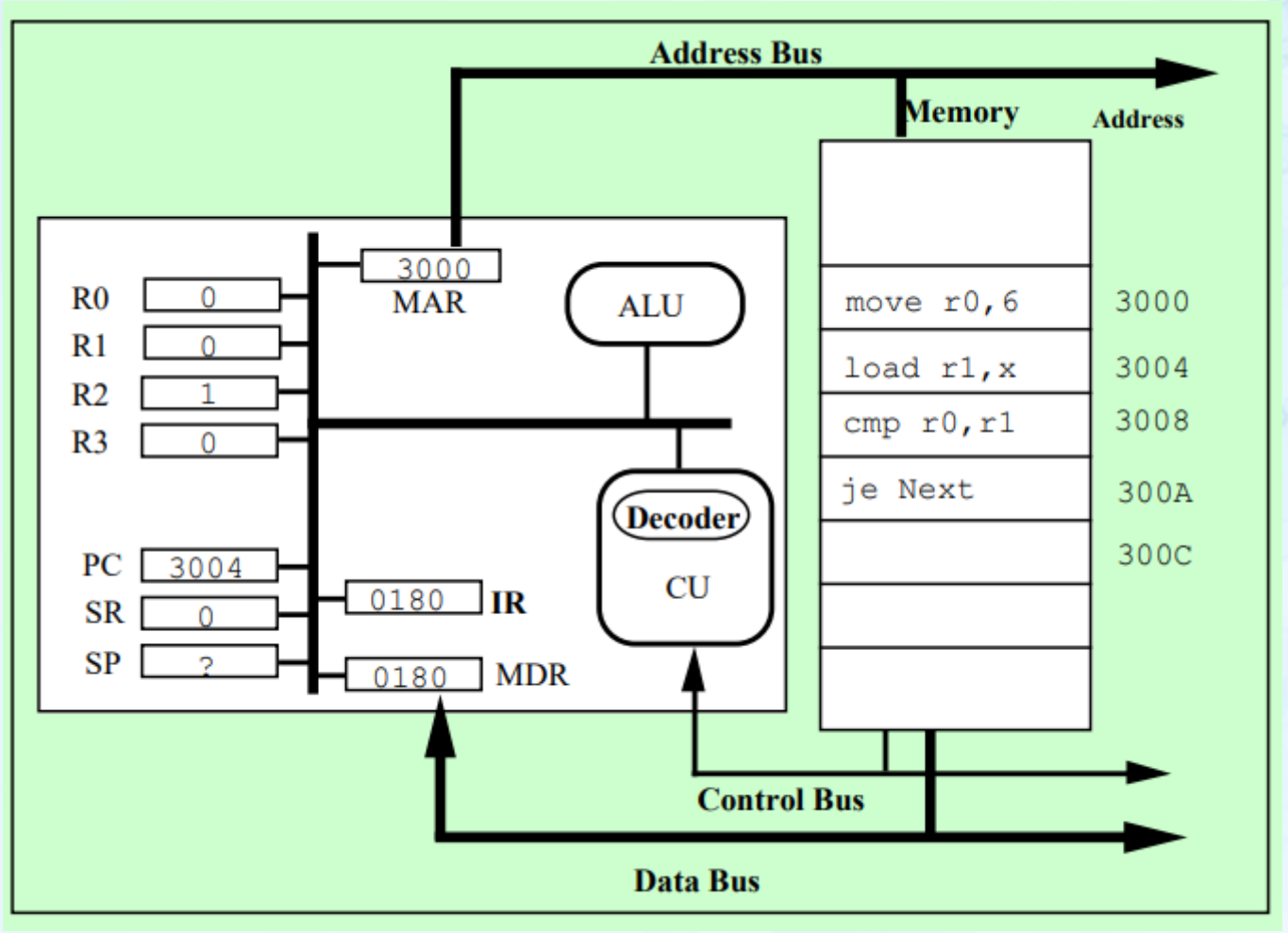
Instruction Life Cycle (How?)



- A program counter is **a special register in a computer processor** that contains the memory address (location) of the next program instruction to be executed.

EELU

Instruction Life Cycle (How?)



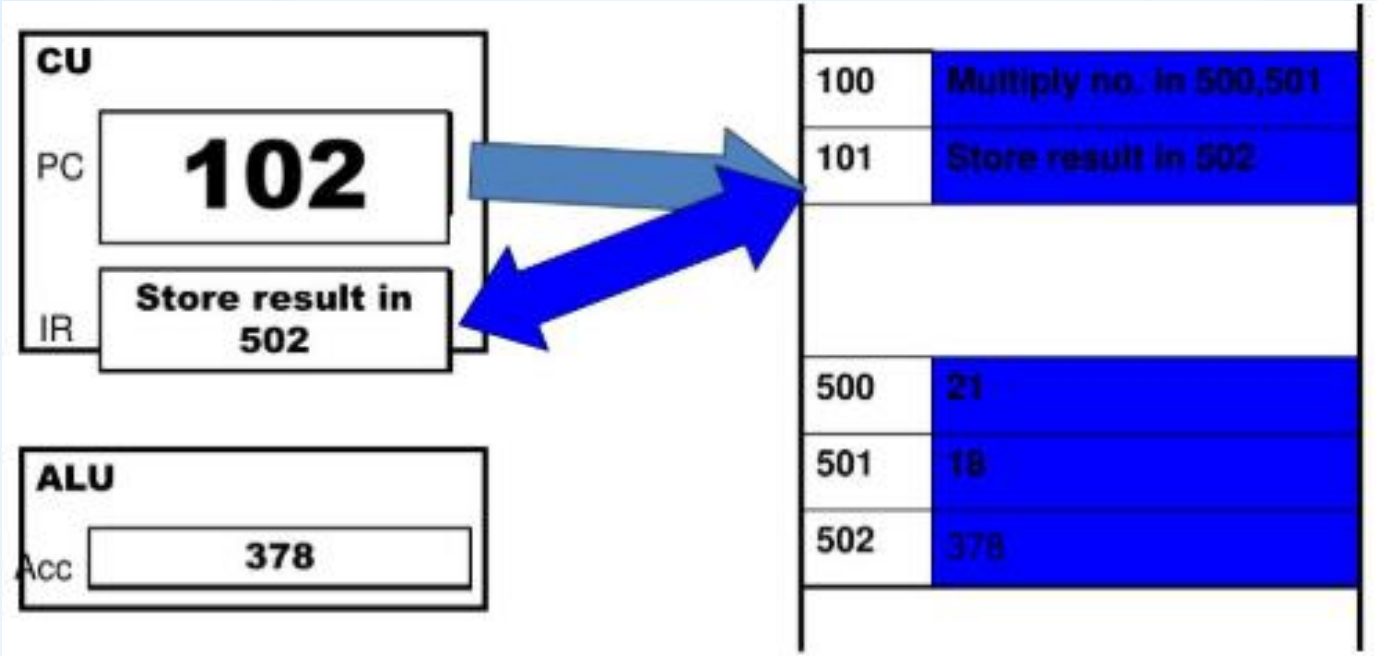
Instruction Life Cycle (How?)

1. Fetch

- Before the CPU fetches an instruction from memory, it references the program counter for the correct memory address.
- After the CPU fetches the instruction, it **increases** the program counter **by 1** so it points to the next instruction in the program's sequence.
- **Next Instruction Not Current Instruction .**

EELU

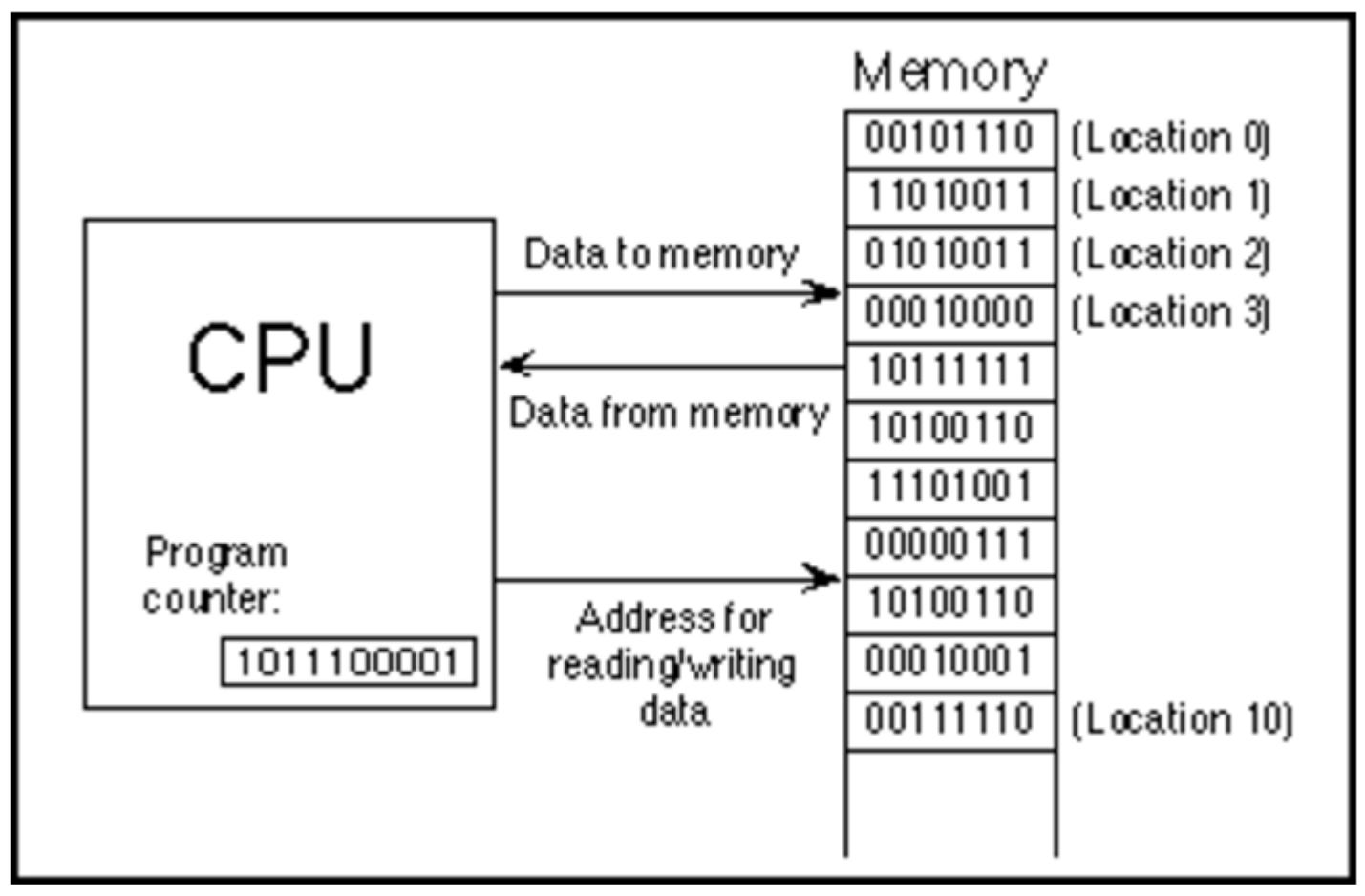
Instruction Life Cycle (How?)



1. The PC contains the address of location 101
2. CU fetches instruction in location 101
3. A copy of the instruction is saved in the IR
4. Increment the PC
5. Activate the right circuits to execute the instruction

EELU

Instruction Life Cycle (How?)

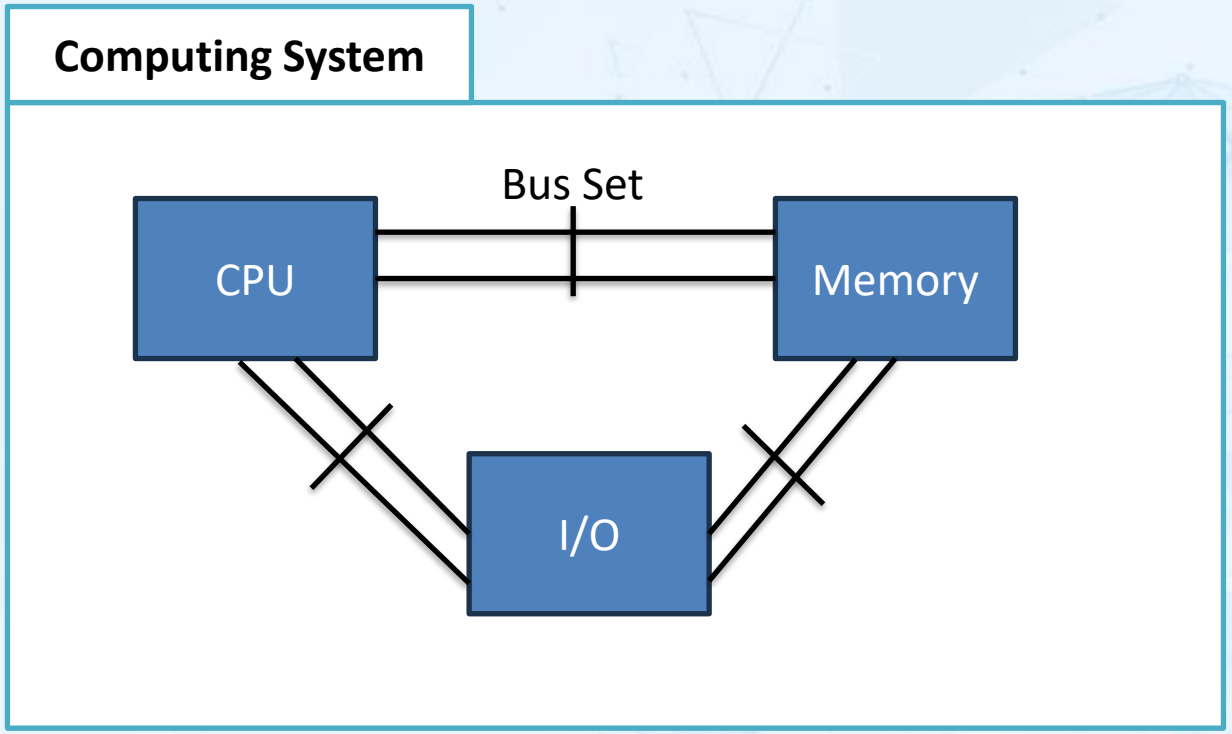


Control Unit (CU)

- Who **fetch** the instruction?
  - The control unit.
- CU fetch the instruction from Memory (outside Microprocessor) to the Instruction Register (IR) – (inside microprocessor to deal with it).

EELU

Why to fetch from outside the CPU to inside CPU?



- Because the connection between the memory and CPU is using **Bus**.
- And it is hard to deal with **Bus**, so the easy way is to fetch from memory to register inside CPU.

EELU

Instruction Life Cycle (How?)
2. Decode

- Instruction Decoder (ID) inside CU is used to decode instruction.
- What is meant by decoding the instruction?
  - Is to understand the instruction to send it to execution.
  - Then to send a **signal** to ALU. (this instruction is Add, Sub, …)

Instruction Life Cycle (How?)
2. Decode

- We need to know **two information** to understand (Decode) the instruction:
  - Instruction Set.
  - Instruction Format.

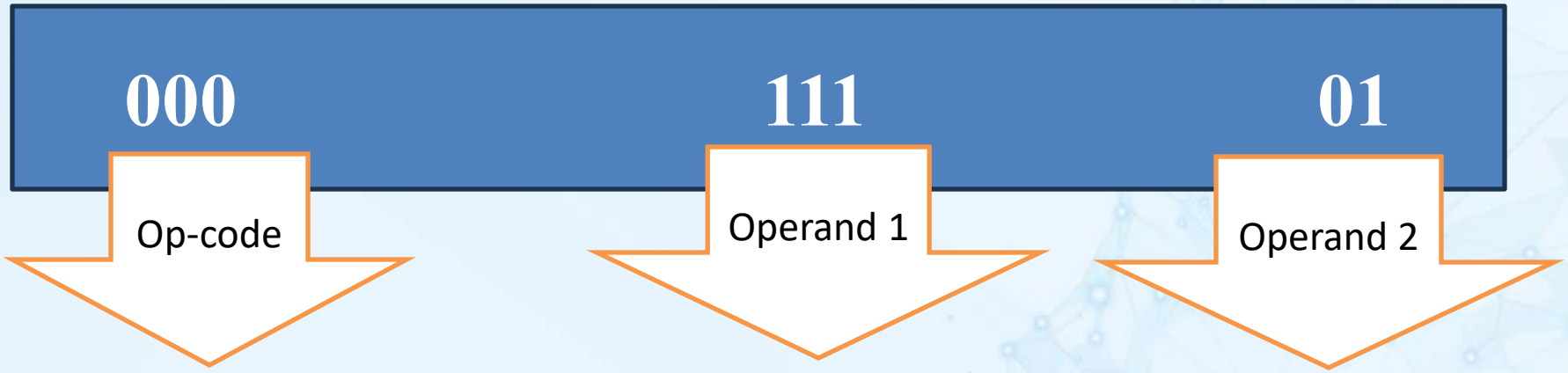
EELU

Instruction Life Cycle (How?)

2. Decode

– Instruction Format: (8-bits , 32-bit, …)

Ex. 8-bit format



– So, **ID** will send to ALU that the operation is ADD, then **ID** will send operand 1 and operand 2.
– Operand 1 : (111) is 7
– Operand 2 : (01) is 1
– **ALU** will execute the results: Add 7 , 1 = 8

EELU

Instruction Life Cycle (How?)
2. Decode

- The result will be stored in Register files.
- Which register?
  - In old processors: Accumulator is register that stores the results of ALU.
  - In new microprocessors: any empty register in general purpose registers stores the results of ALU.

EELU

Instruction Life Cycle (How?)

2. Decode

- Instruction Set: you can understand that the instruction contains ADD operation using Instruction Set.

Normal instructions	
00000	SUB
00001	AND
00010	ADD
00011	OR
00100	XOR

Note that:

If we have two microprocessors

Microprocessor 1	Microprocessor 2
This microprocessor understand that 000 means **add** operation	This microprocessor understand that 111 means **ADD** operation
Each microprocessor have different machine language to understand.	
So, the assembly language will vary depending on different microprocessors.	
Based on that, using C language is better than using assembly language.	
And using high performance processors / complier, the advantage of assembly language (speed/ faster than C) is not important now.	

EELU

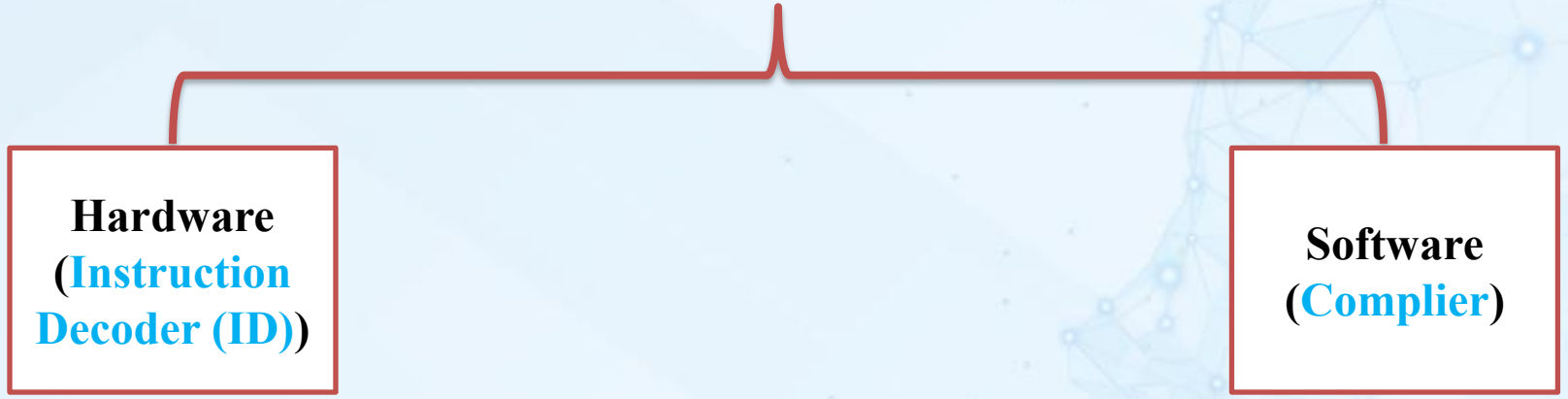
Note that:

If we have two microprocessors

Microprocessor 1	Microprocessor 2
This microprocessor understand that 000 means **add** operation	This microprocessor understand that 111 means **ADD** operation
The instruction is written **add**	The instruction is written: **ADD**
Microprocessor 1 understand (- , +) only	Microprocessor 2 Understand (- , + , * , /)
If the instruction is * in microprocessor 1 Then the **compiler (SW)** will convert it into many add operations.	

To understand the Instruction

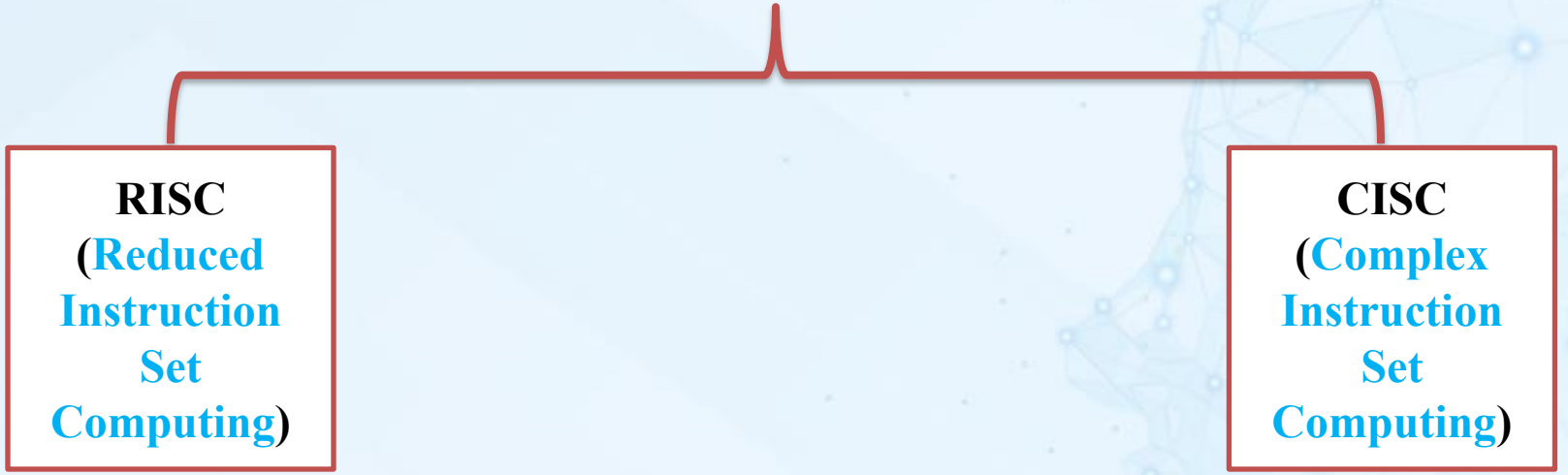
- We must have two things: (Hardware & Software).



EELU

Instruction Set Architecture (ISA)

- ISA has two main types: (RISC & CISC).



EELU

Instruction Set Architecture (ISA)

Points	RISC	CISC
No. of Instructions	Understand **less** instructions	Understand **many** instructions
Software (Compiler)	Strong	Simple

- Why RISC need Strong SW?
- Because if it only understand (+ & -), then the instruction is * , so it needs strong compiler to convert * into many add operations.

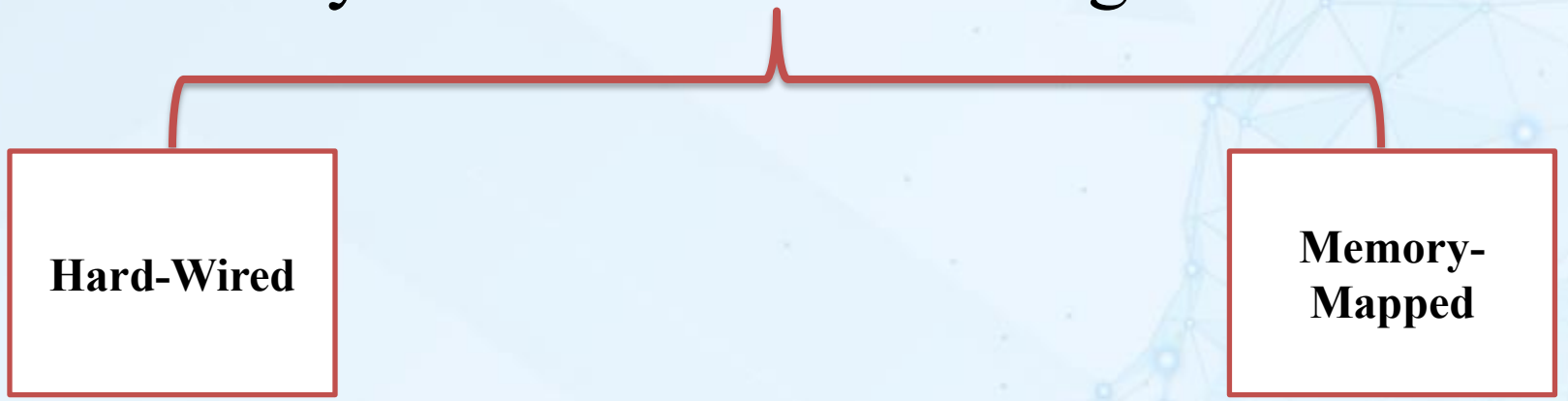
EELU

Hardware

- Instruction Decoder (ID), is the hardware that understand the instruction.
- ID manufacturing methods vary depending on the (Instruction Set Architecture) ISA, it is RICS or CISC.

Hardware

- There are two ways for ID manufacturing.



Hardware

- **Hard-Wired:**

Hard-Wired

- Hard-Wired consists of set of logic gates, so ID is fast.
- It works as (Input → circuit gate → output), so it is fast.
- If we need to ID to understand **many** instructions, then increase the logic gates. But it is expensive as it is a hardware.
- Need strong compiler.
- RICS (less instructions)

EELU

Hardware

- **Memory-Mapped:**

Memory-Mapped

- We store instructions on it.
- It works by searching for instructions in memory. So, **it takes time**.
- Need simple compiler (not strong) , as every instructions could be found. Ex (+ , - , * , /), then we don't need conversion.
- CICS

EELU

Instruction Set Architecture (ISA)

ES Challenges	RISC	CISC
Size	ID: Bigger ---------- ALU: small (understand less commands)	ID: Smaller ---------- ALU: Big (understand many commands)
Cost	SW (Compiler) – high ---------- HW (ID) – understand less commands so, it need less logic gates – low	SW (Complier) – Lower ---------- HW (ID) - High

EELU

Instruction Set Architecture (ISA)

ES Challenges	RISC	CISC
Performance	Time in conversion	Time in search
Power consumption	ALU: less (understand less commands) ID: high(made from logic gates)	ALU: high ID: less

- So, both are the same.
- And based on what we need, RICS or CISC.

Instruction Set Architecture (ISA)

- In Embedded systems we use RISC, as we need less no. of instructions.
- CISC is a high performance like computers.

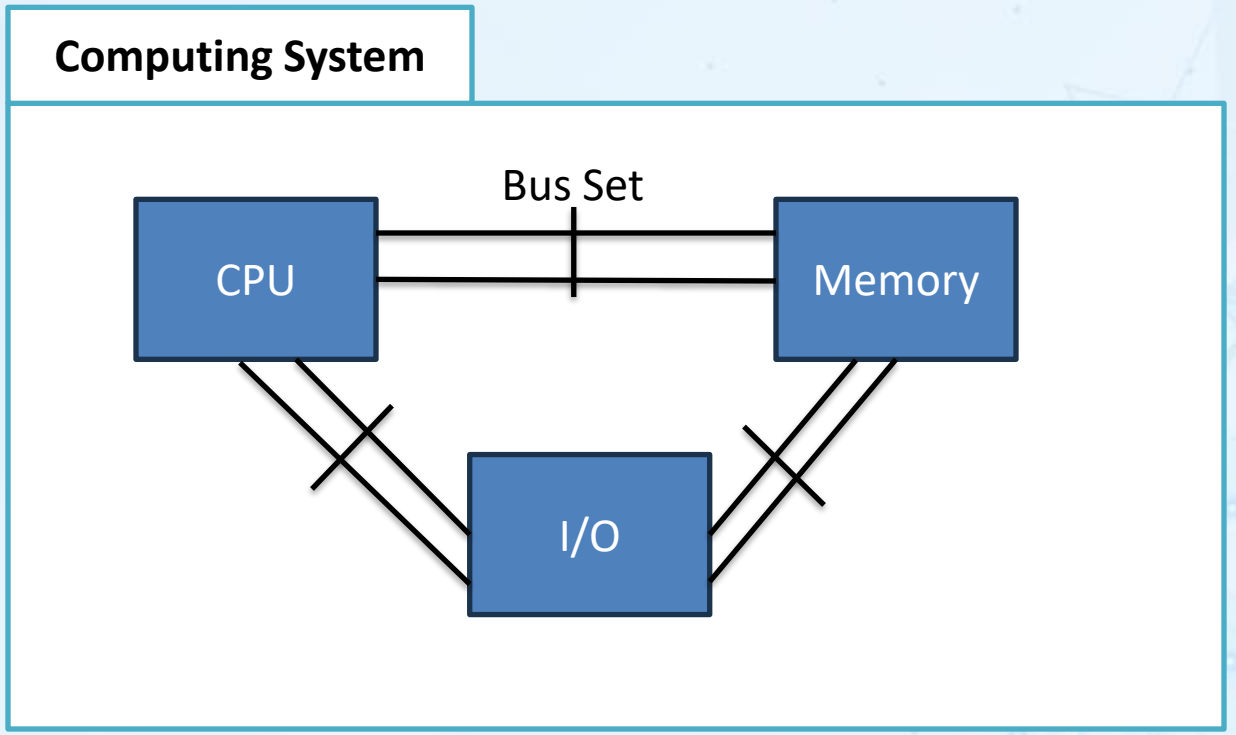
RISC (Reduced Instruction Set Computing)

- RISC is designed to perform smaller number of types of computer instructions so that it can operate at higher speed.
- The range of instruction is 30 to 40.
- Since each instruction type (that a computer must perform) requires additional transistors and circuitry a large list or a set of computer instruction tends to make microprocessor more complicated and slower in operation.
- It is a type of microprocessor architecture that utilizes a small highly optimized set of instruction rather than a more specialized set of instruction often found n other types of architecture.
- With Harvard Architecture.

EELU

CISC (Complex Instruction Set Computing)

- A complex instruction set computer (CISC ‘sɪsk’) is a computer in which single instructions can execute several low-level operations (such as a load from memory, an arithmetic operation, and a memory store) or are capable of multi-step operations or addressing modes within single instructions.
- The term was retroactively coined in contrast to reduced instruction set computer (RISC) and has therefore become something of an umbrella term for everything that is not RISC, from large and complex mainframe computers to simplistic microcontrollers where memory load and store operations are not separated from arithmetic instructions.
- Can be Harvard or Von-Neumann Architecture.


Computing System
Bus Set
CPU
Memory
I/O

Register Files

- There are two types of Register files:
  - (General Purpose Registers) GPR
  - (Special Purpose Registers) SPF/ (Special Functions Registers) SFR

Register Files

- General Purpose Registers (GPR):
  - Data is temporarily stored.
  - Ex. To store ALU results temporarily until CU store it finally in memory again (Data memory “RAM”).
  - In C language, to create datatype and put **register** keyword before it, it means to store in GPR to be near to CPU and to be easily for microprocessor to access it many times. Then it will be faster for microprocessor.

EELU

Register Files

- General Purpose Registers (GPR):
  - Are you sure while putting **register** keyword, it always means to store in GPR??
  - No.
  - Based on the registers, if the compiler uses it now or not (available or not).
  - If (not available), so it will be stored in memory and to deal with it there.

Register Files

- Specific Purpose Registers (SPR):
  - Each register has specific purpose.
  - Each register has a specific function only for it.

1. Program Counter (PC)
   1. Placed in ROM.
   2. Hold the address (location) of next instruction.
2. Stack Pointer (SP)
   1. It is a small register that stores the memory address of the last data element added to the stack.
   2. The stack is a specific region of memory that is used to store temporary data in a LIFO (Last In, First Out) manner. It is part of the overall memory structure in a computer, which is divided into various sections, each serving different purposes.
   3. SP Hold the last place in the stack. (top)

Register Files

- Specific Purpose Registers (SPR):

  3. Accumulator (ACC):

     - store results from ALU temporarily.

     - nowadays it is unavailable.

  4. Instruction Register (IR):

     - stores instruction inside it, after fetch from memory.

EELU

الجامعة المصرية للتعلم الإلكتروني الأهلية

THE EGYPTIAN E-LEARNING UNIVERSITY

THANK YOU FOR WATCHING

QUESTIONS?

